

Object Oriented Programming

Dr. Imran Ashraf

Office: 02

`imran.ashraf[at]hitecuni[dot]edu[dot]pk`

Last Week

- Inheritance
 - Motivation
 - Class diagram
 - Example

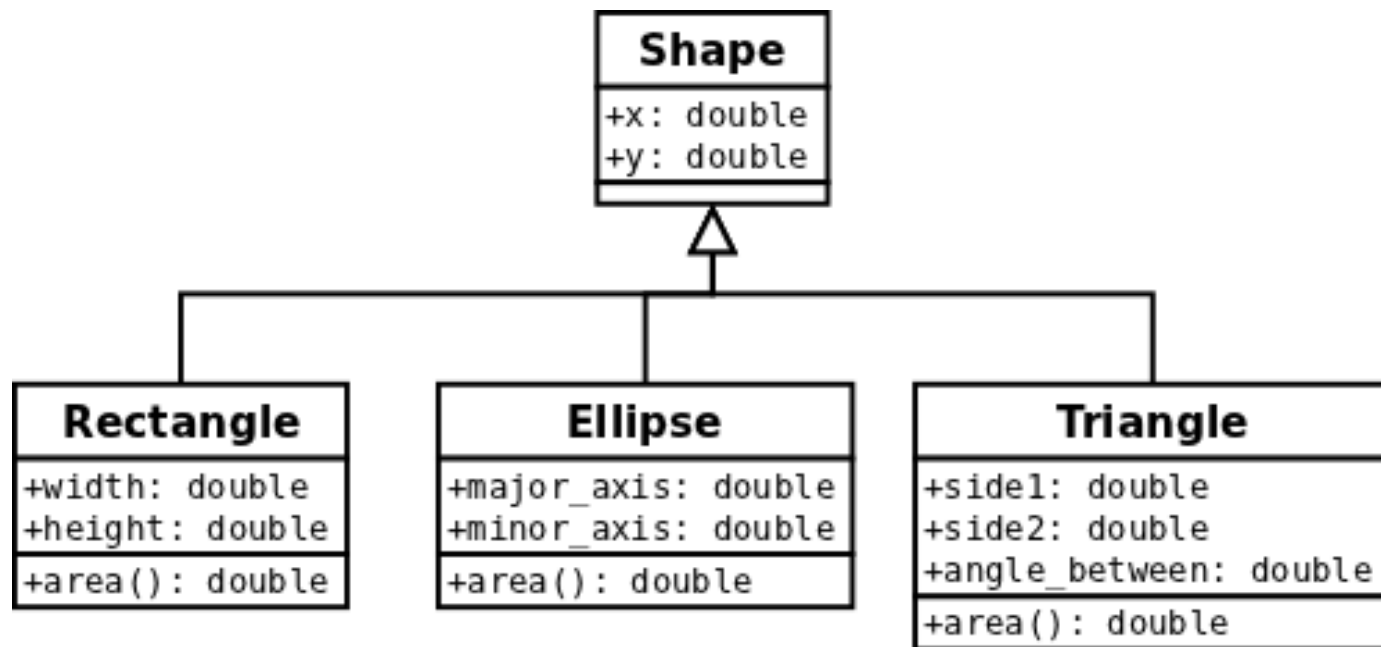
This Week

- Project
- Recap
- Overloading vs. Overriding
- Levels of Inheritance
- Multiple Inheritance

Recap

- Real world examples
 - Shape (square, circle, triangle ...)
 - Person (Dean, HoD, Lecturer, Student)
 - Vehicle (Bus, Truck, Car ...)

Recap



Overriding

- use member functions in a derived class that override—that is, have the same name as—those in the base class
- calls in your program work the same way for objects of both base and derived classes
- How does the compiler know which one to use?
- Scope Resolution with overridden functions?

Access Combinations

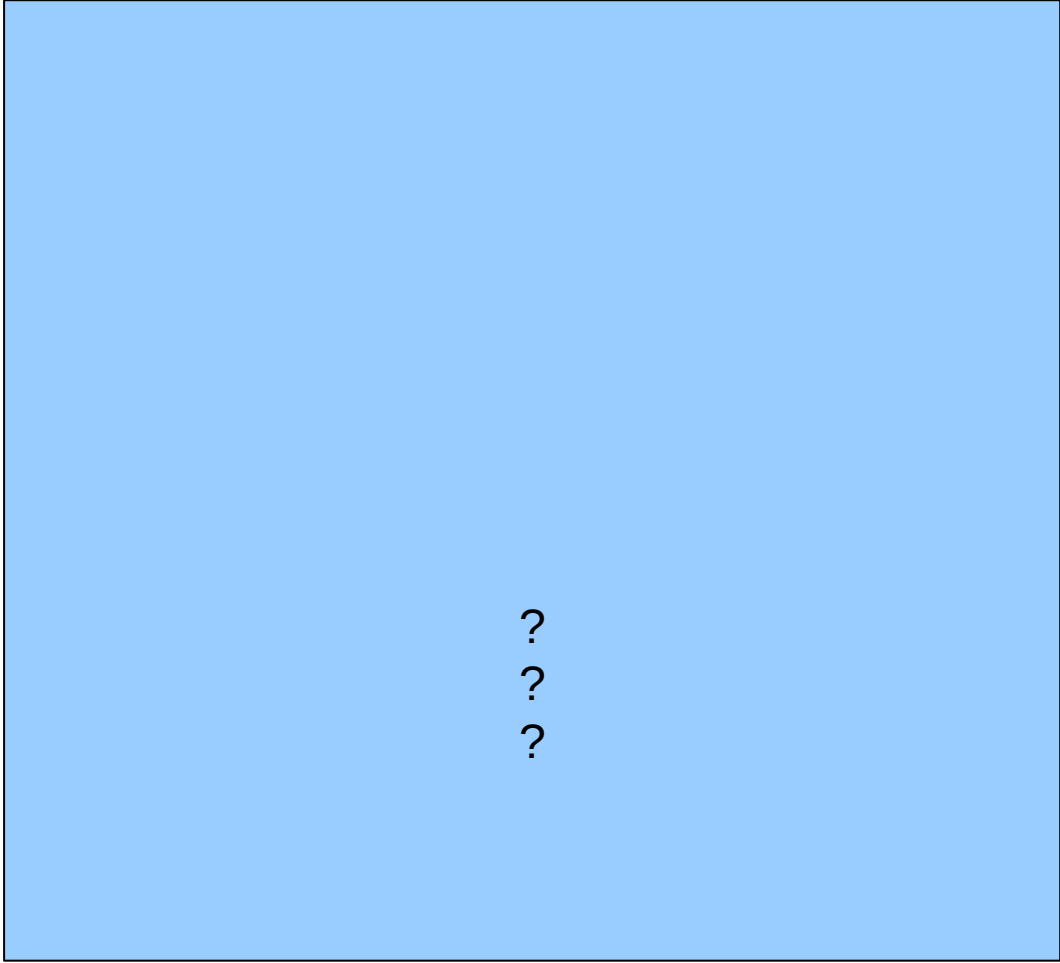
- C++ provides a wealth of ways to fine-tune access to class members.
- One such access-control mechanism is the way derived classes are declared.
 - Public
 - Protected
 - Private

Access Combinations: Example

```
class A                                //base class
{
    private:
        int privdataA;                //(functions have the same access
protected:                            //rules as the data shown here)
        int protdataA;
public:
        int pubdataA;
};
```


Access Combinations: Example

```
class B : public A
{
public:
    void funct()
    {
        int a;
        a = privdataA;
        a = protdataA;
        a = pubdataA;
    }
};
```



?

?

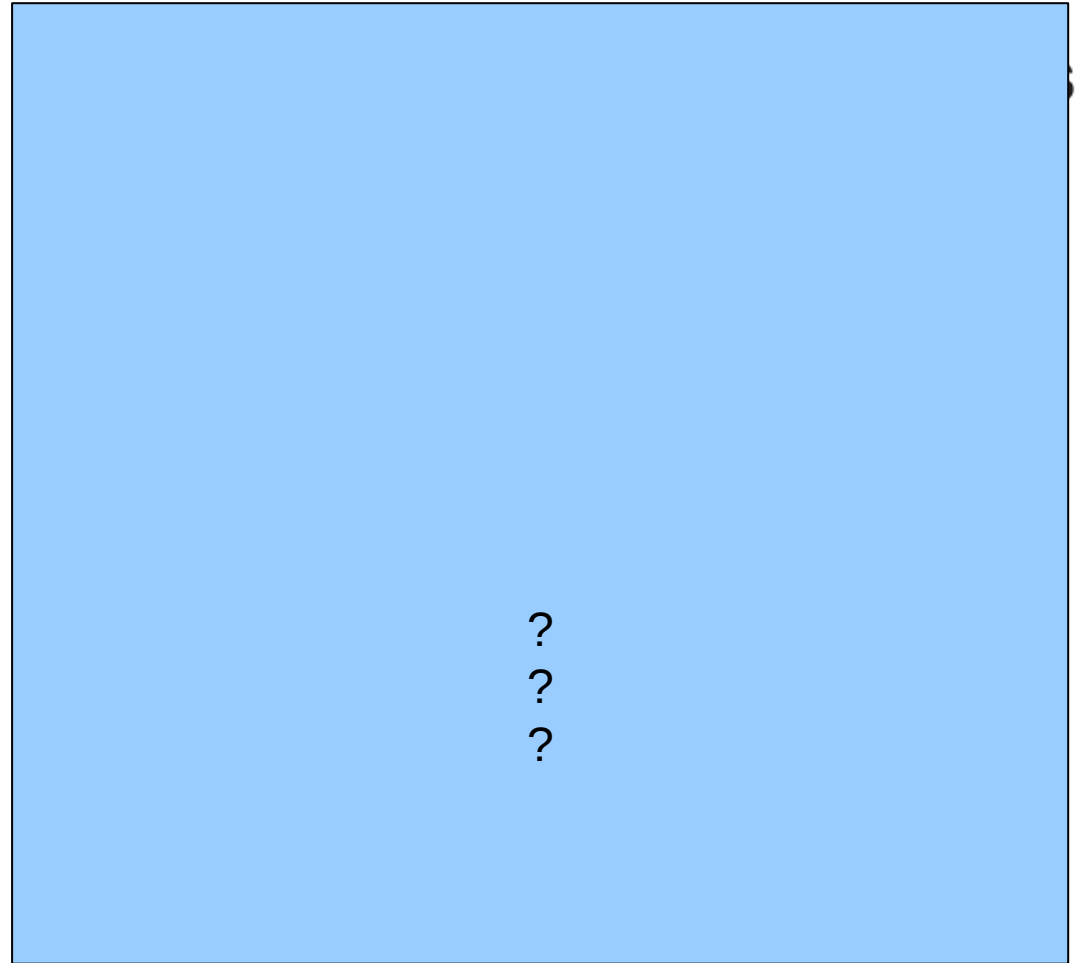
?

Access Combinations: Example

```
class B : public A           //publicly-derived class
{
public:
    void funct()
    {
        int a;
        a = privdataA;      //error: not accessible
        a = protdataA;      //OK
        a = pubdataA;       //OK
    }
};
```

Access Combinations: Example

```
class C : private A
{
public:
    void funct()
    {
        int a;
        a = privdataA;
        a = protdataA;
        a = pubdataA;
    }
};
```



Access Combinations: Example

```
class C : private A          //privately-derived class
{
public:
    void funct()
    {
        int a;
        a = privdataA;      //error: not accessible
        a = protdataA;      //OK
        a = pubdataA;       //OK
    }
};
```

Access Combinations: Example

```
B objB;  
a = objB.privdataA;  
a = objB.protdataA;  
a = objB.pubdataA;
```

```
C objC;  
a = objC.privdataA;  
a = objC.protdataA;  
a = objC.pubdataA;
```

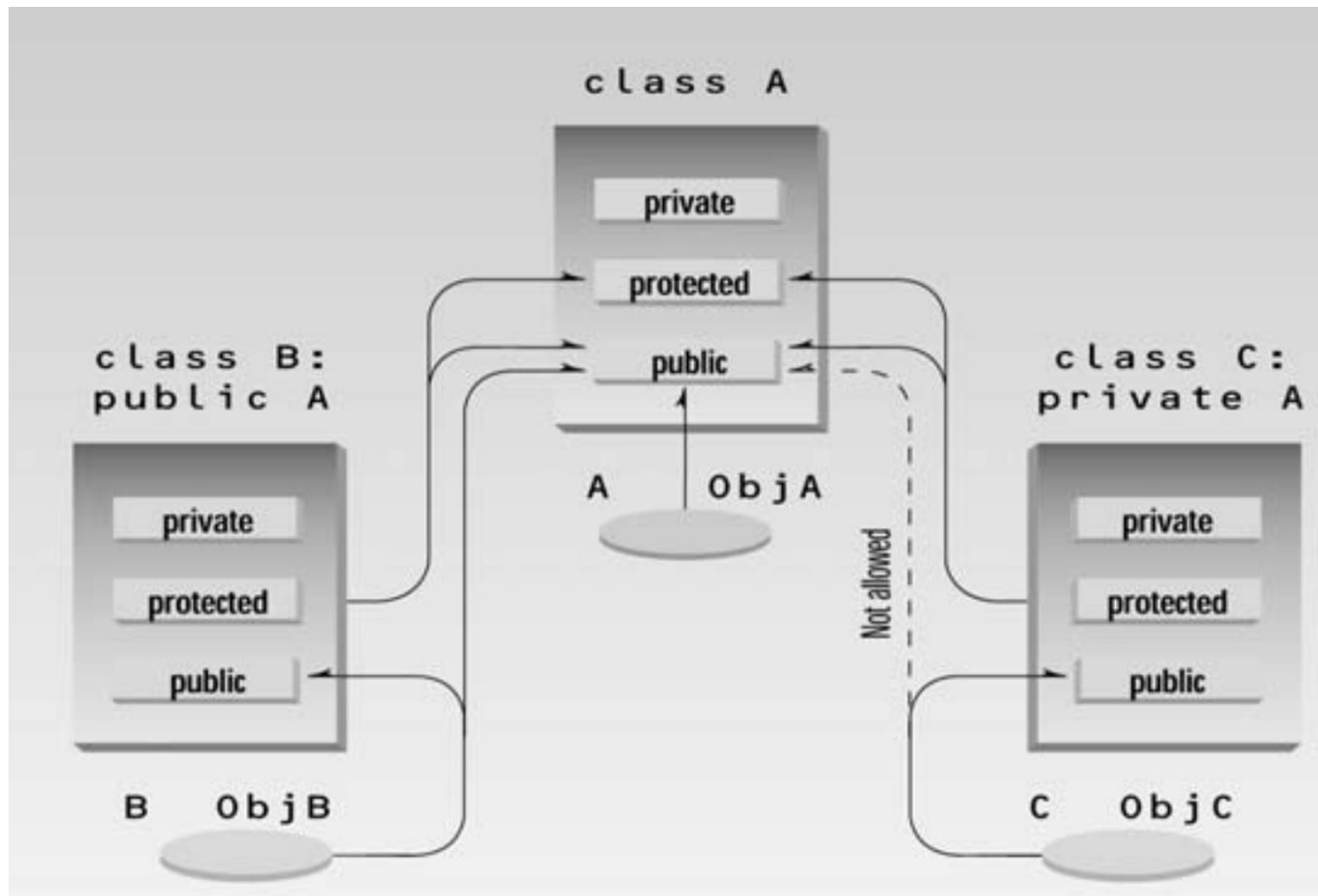
?

?

Access Combinations: Example

```
B objB;  
a = objB.privdataA;    //error: not accessible  
a = objB.protdataA;    //error: not accessible  
a = objB.pubdataA;     //OK (A public to B)  
  
C objC;  
a = objC.privdataA;    //error: not accessible  
a = objC.protdataA;    //error: not accessible  
a = objC.pubdataA;     //error: not accessible (A private to C)
```

Access Combinations: Example



Access Combinations:

When to use what?

- Derived class offers specialized or improved version of base class, so public should be used
 - CountDn and Manager classes were specialized versions of Counter and Employee classes, respectively

Access Combinations:

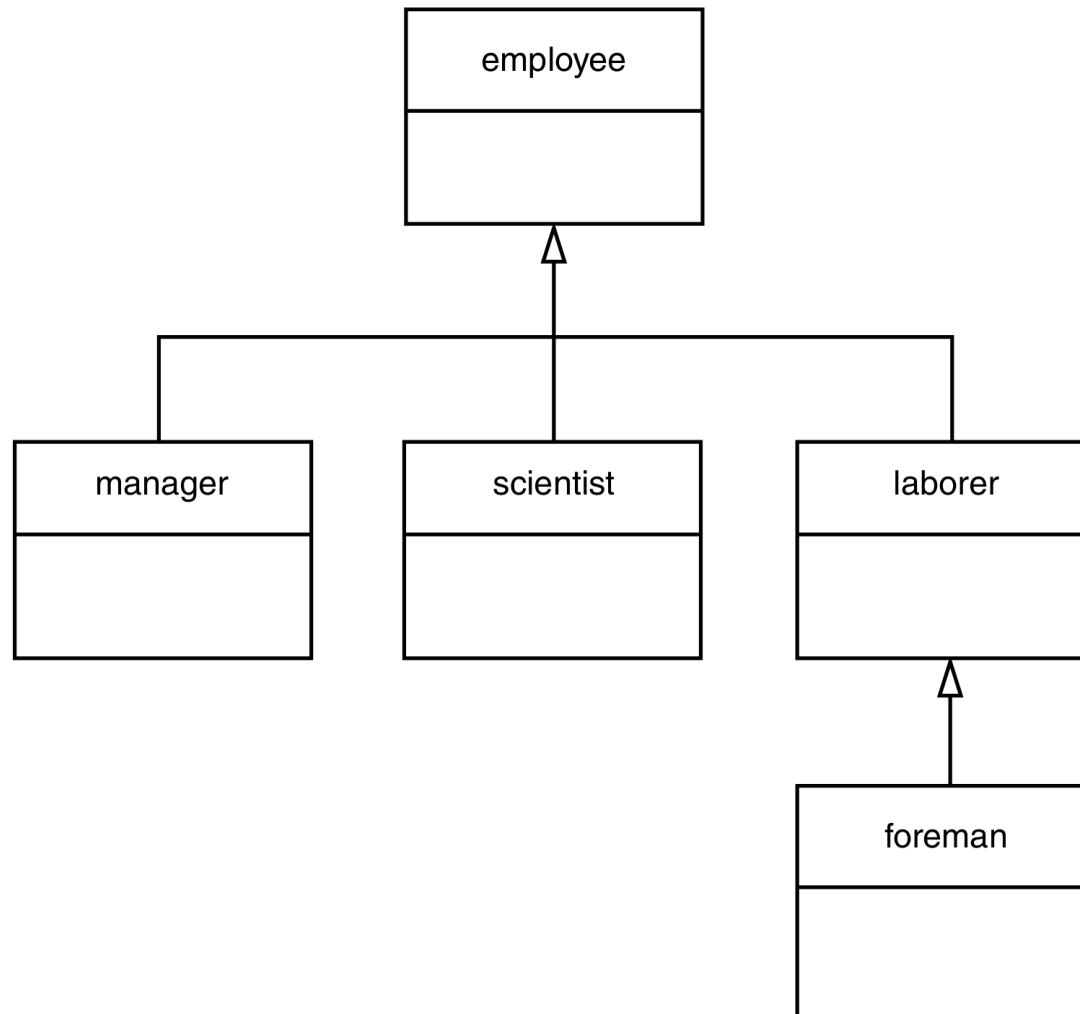
When to use what?

- In cases where derived class completely modifies the operation of the base class or hides or disguises its original interface, so private should be used
 - Stack can be derived from Array with the private

Levels of Inheritance

```
class A
    { };
class B : public A
    { };
class C : public B
    { };
```

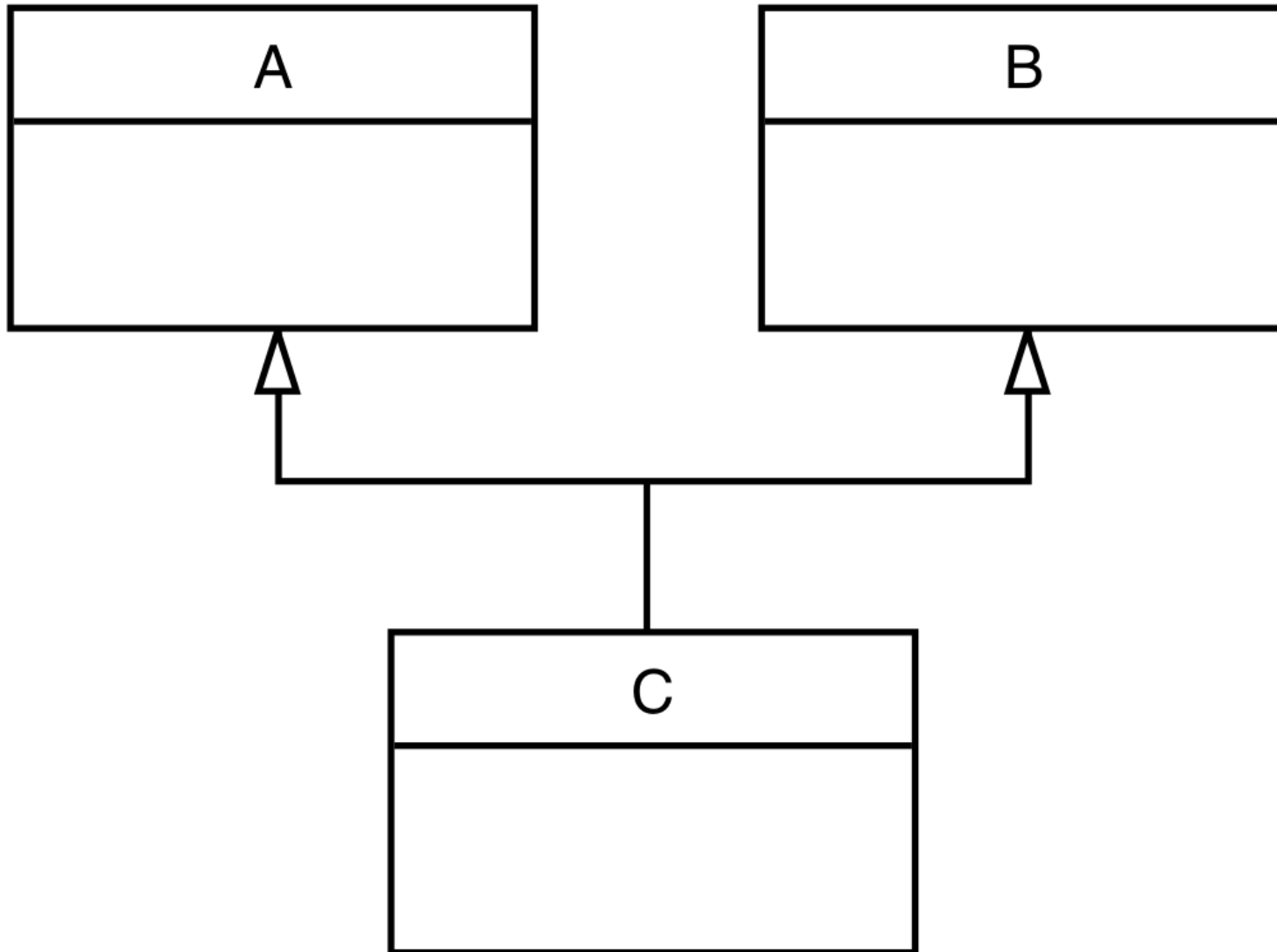
Multi-level Inheritance



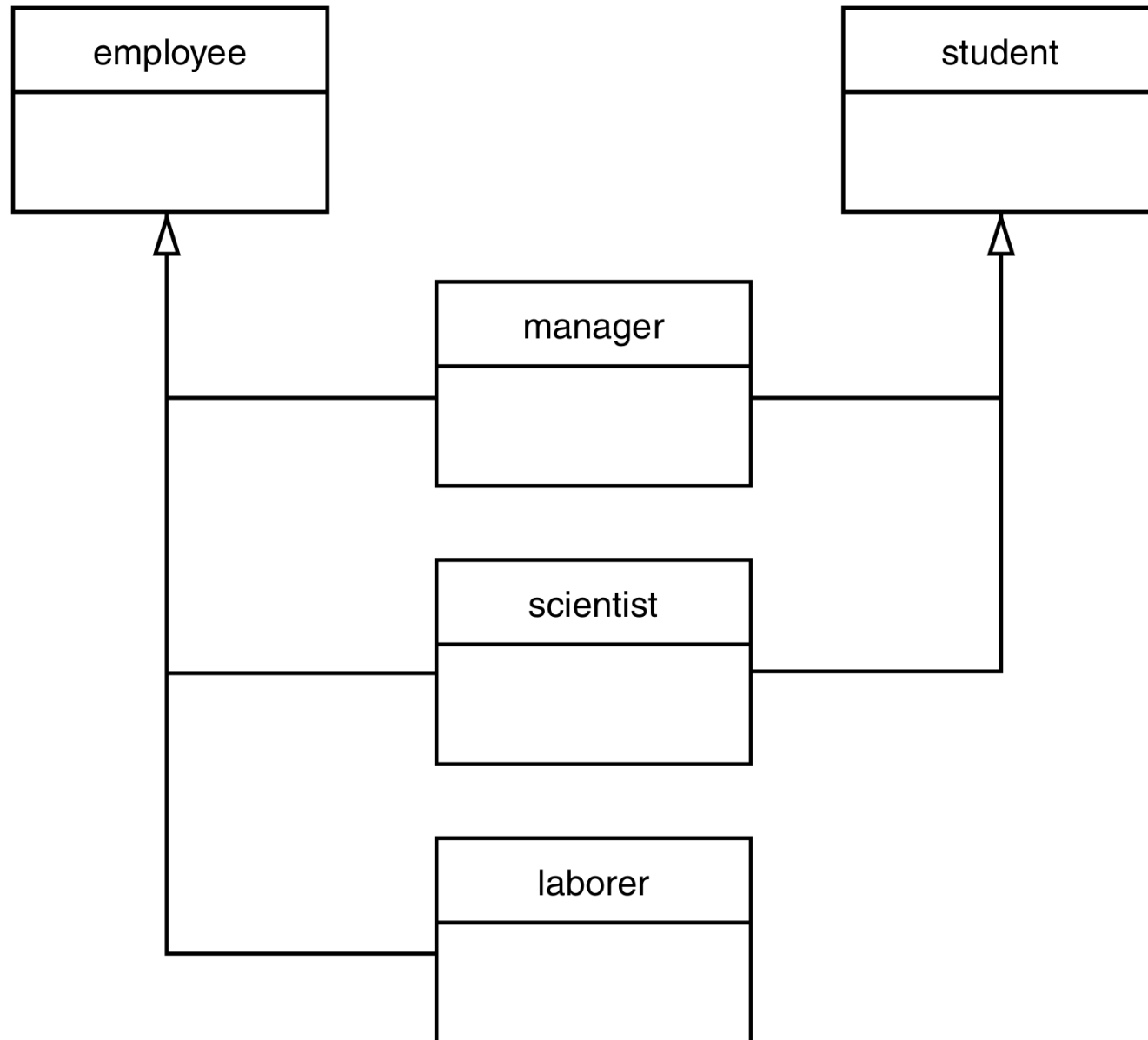
Foreman Class definition

```
.....
class laborer : public employee    //laborer class
{
};
////////////////////////////////////
class foreman : public laborer    //foreman class
{
private:
    float quotas; //percent of quotas met successfully
public:
    void getdata()
    {
        laborer::getdata();
        cout << "    Enter quotas: "; cin >> quotas;
    }
    void putdata() const
    {
        laborer::putdata();
        cout << "\n    Quotas: " << quotas;
    }
};
```

Multiple Inheritance



Multiple Inheritance



Multi-level Inheritance

```
class manager : private employee, private student //management
{
private:
    char title[LEN];          //"vice-president" etc.
    double dues;              //golf club dues
public:
    void getdata()
    {
        employee::getdata();
        cout << "    Enter title: ";          cin >> title;
        cout << "    Enter golf club dues: "; cin >> dues;
        student::getedu();
    }
    void putdata() const
    {
        employee::putdata();
        cout << "\n    Title: " << title;
        cout << "\n    Golf club dues: " << dues;
        student::putedu();
    }
};
```

Summary

- Inheritance is an important C++ concept
- Understanding access combination is important
- Multiple inheritance is also possible in C++
- Inheritance can be multi-level